



東華大學
DONGHUA UNIVERSITY

图像数据处理

DATA

计算机科学与技术学院

主讲教师：李柏岩、宋晖

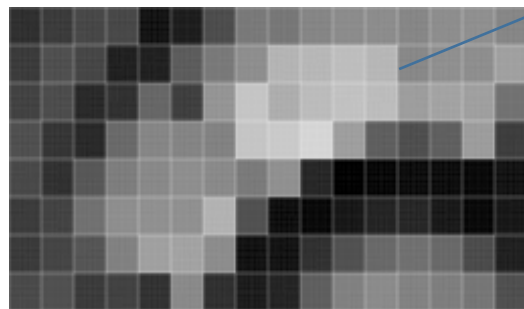
图像数据处理

- 图像是人类获取信息、表达信息和传递信息的重要手段
- 早期：使用计算机绘制图像，建立三维模型
- 人工智能技术发展，计算机开始试图自动识别图像的内容
 - 手写数字识别
 - 车牌识别
 - 人脸识别
 -

数字图像

- 将图像进行数字化
 - 使用给定大小的网格将连续图像离散化,
 - 每个小方格（像素）被记录为一种颜色
 - 颜色矩阵表示数字图像
- 像素（Pixel）
 - 数字图像的最小单位
 - 每个像素具有横和纵位置坐标，以及颜色值

像素也被用来表示整幅图像的网格数，如 640×480
同样大小的图像，像素越大越清晰



	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214
75	23	16	21	34	59	45	12	12	11	45	73	81	94	95	86	86
76	17	27	31	47	33	14	20	41	78	81	96	108	108	98	98	109
77	27	27	54	45	14	17	51	90	115	162	153	170	148	113	109	109
78	27	41	44	13	37	62	52	105	174	144	161	173	159	128	130	130
79	35	41	33	24	60	105	90	100	173	179	198	123	71	38	69	124
80	49	41	27	60	86	102	113	102	88	104	26	0	8	6	5	5
81	46	34	39	72	113	115	110	144	49	16	6	6	17	27	16	4
82	42	32	40	58	92	123	138	110	6	6	29	54	73	75	65	46

(a) 灰度图像

(b) 图像局部 (16×8) 像素图

图像局部 (16×8) 的像素矩阵

数字图像类型 (1)

- 二值图像
 - 像素矩阵由0、1两个值构成，
 - “0” 代表黑色，“1” 代白色，用1位二进制表示
 - 通常用于文字、线条图的扫描识别（OCR）和掩膜图像的存储
- 灰度图像
 - 灰度图像矩阵元素的取值范围通常为[0, 255]
 - “0” 表示纯黑色，“255” 表示纯白色，中间的数字从小到大表示由黑到白的过渡色
 - 用8位二进制表示
 - 二值图像可以看成是灰度图像的特例

数字图像类型 (2)

- RGB彩色图像

- 每个像素的颜色表示为：红 (R)、绿 (G) 和蓝 (B) 三原色组合
- 图像用3个 $M \times N$ 的二维矩阵
 - 每个矩阵分别存放一个颜色分量，取值范围在 $[0, 255]$ ，表示该原色在该像素的深浅程度
- 每个像素的颜色使用 $3 \times 8\text{bit}$ 表示，也被称为24位图。

- 图像压缩

- 直接存储数字图像的二维矩阵存储非常大，通常将原始数据压缩后进行存储
- 常用格式
 - BMP、JPEG、TIF、GIF、PNG等

数字图像处理 (1)

- 图像变换 (Geometrical Image Processing)
 - 几何变换
 - 坐标变换, 图像的放大、缩小、旋转、移动, 多个图像配准, 全景畸变校正, 扭曲校正, 周长、面积、体积计算等
 - 空间变换
 - 傅里叶变换、离散余弦变换, 小波缓缓将图像从时域变换到频域
- 图像增强和复原 (Image Enhancement & Restoration)
 - 目的: 提高图像的质量
 - 方法: 去除噪声, 提高图像的清晰度

数字图像处理 (2)

- 图像重建 (Image Reconstruction)
 - 通过物体外部测量的数据经数字化处理获得物体的三维形状信息
 - 投影重建、明暗恢复形状、立体视觉重建和激光测距重建
- 图像编码 (Image Encoding)
 - 利用图像的统计特性、人类视觉生理学及心理学特性对图像数据进行编码
 - 以较少的比特数表示图像或图像中所包含的信息
 - 常见有JPEG、TIFF等压缩格式

数字图像处理 (3)

- 图像识别 (Image Recognition)
 - 利用计算机对图像进行处理、分析和理解, 识别各种不同模式的目标和对象空间变换
 - 广泛地应用于导航、地图与地形配准、自然资源分析、天气预报、环境监测、生理病变研究等领域。

7.2 Python图像处理

- Python图像处理库
 - 常用：PIL、Pillow、OpenCV以及Scikit-image等

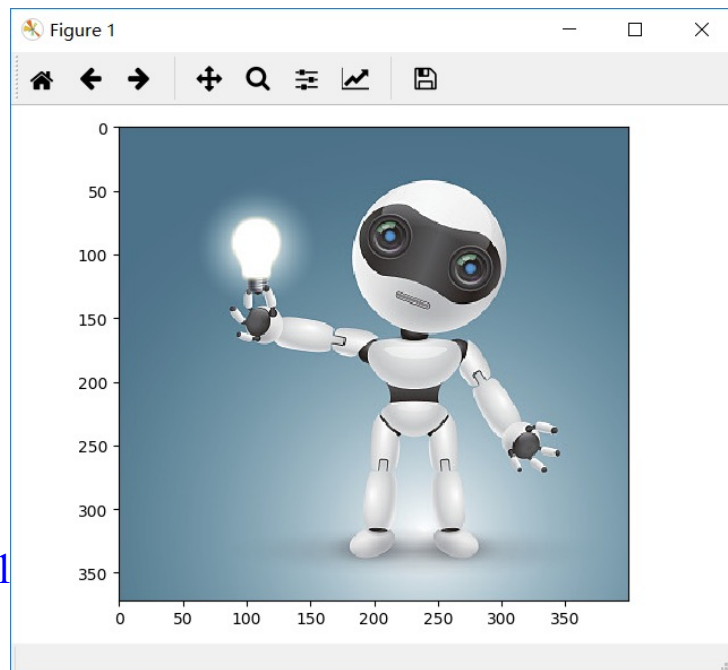
表7-1 Scikit-image的常用模块

子模块名称	主要实现功能
io	读取、保存和显示图片或视频
data	图片和样本数据
color	颜色空间变换
filters	图像增强、边缘检测、排序滤波器、自动阈值等
transform	几何变换或其它变换，如旋转、拉伸和拉东变换等
feature	特征检测与提取等
measure	图像属性的测量，如相似性或等高线等
segmentation	图像分割
restoration	图像恢复
util	通用函数

图像基本操作 (1)

- 图像读取和显示
 - 用ndarray的多维数组表示图像
 - Scikit-image库的io库
 - 图片输入输出

```
>>> robot = io.imread("data/Robot.jpg")
>>> robot.shape    # 图像像素和颜色字节数
(372, 400, 3)
>>> type(robot)    # 数据类型
<class 'numpy.ndarray'>
>>> io.imshow(robot)
<matplotlib.image.AxesImage object at 0x22099dd21...
>>> io.show()
```



图像基本操作 (2)

- 图像的坐标和颜色

- 使用 (row, col) 表示图像每个像素的坐标
- 起点 (0,0) 位于图像的左上角
- 给出一个坐标位置, 即可获得图像中该像素的颜色。

```
>>> robot[91,221]    #取指定坐标像素的颜色  
array( [65 61 62], dtype=uint8)
```

- RGB彩色图像

- 每个像素的颜色用一个 (R,G,B) 三元组表示
- 可以只提取某个通道 (分别用0、1、2表示) 的颜色值。

```
>>> robot[91,221, 0]    #取指定坐标像素的R值  
65
```

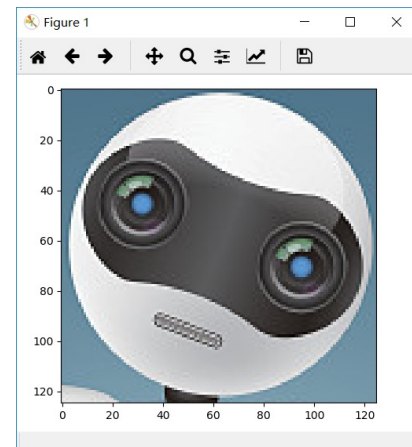
图像基本操作 (3)

- 数组的切片操作
 - 可以访问图像中某一部分的颜色

```
>>> robot[77:80,221:231, 0]    #取一部分图像的R值  
array([[ 37  90  79  61  41  42 129  75  75  72]  
       [ 32  38  85  63  52  41  78 113  65  71]  
       [ 38  33  69  78  60  44  53 116  68  63]], dtype=uint8)
```

- 图像裁剪
 - 提取数组的部分数据显示和保存

```
>>> head = robot[40:165,180:305]  #给出图像局部 head的坐标范围  
>>> io.imshow(head)  
>>> io.show()  
>>> io.imwrite('data/RobotHead.jpg', head) #将图像数据保存为文件
```

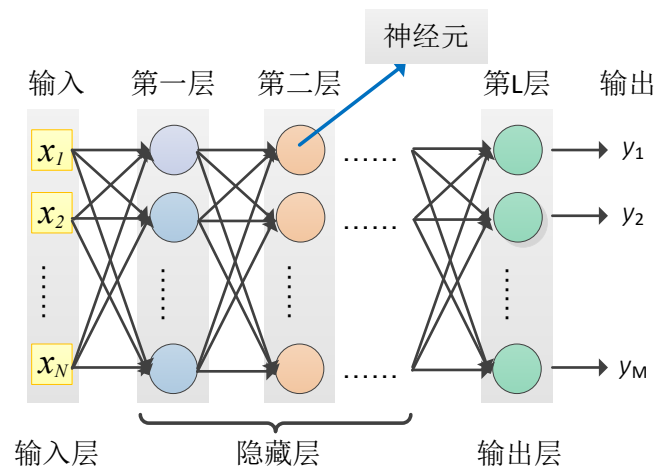


7.3 案例：深度学习实现图像分类

- 图像特征提取
 - 基于色彩特征
 - 基于纹理
 - 基于形状
 - 基于空间关系等
- 深度学习算法
 - 传统前馈神经网络
 - 参数多
 - 计算量大
 - CNN

7.3 案例：深度学习实现图像分类

- 前馈神经网络
 - 全连接



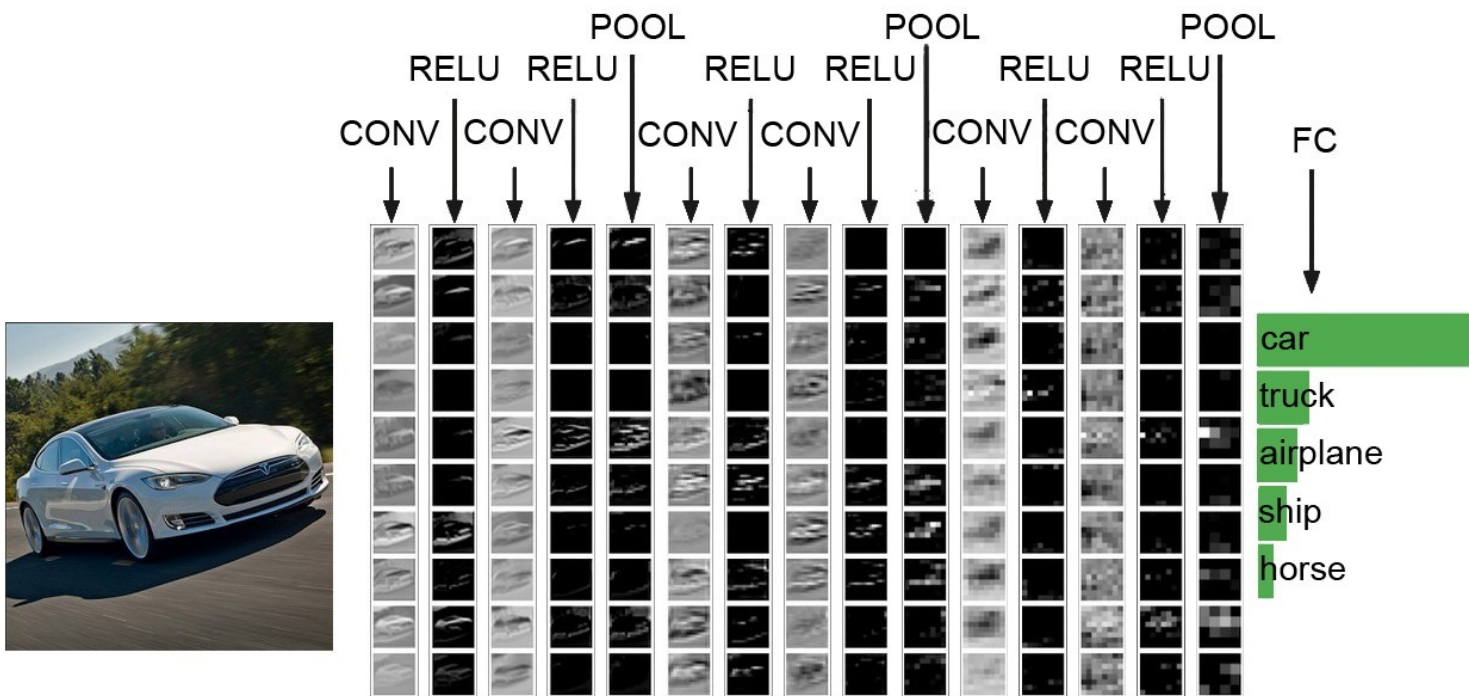
- 图像数据非常大
 - 1000×1000 像素的灰度图像
 - 输入层有 $1000 \times 1000 = 100$ 万个节点
 - 第一个隐层有 1000 个节点
 - 有 $1000 \times 1000 \times 1000 = 1$ 亿个权重参数需要学习

卷积神经网络（CNN）

- 图像每个像素与周围像素关联紧密，与离得远的像素之间关联很小
- CNN网络引入特定隐藏层，减少网络参数
 - 卷积层（Conventional Layer）
 - 由若干卷积单元组成
 - 每个卷积单元仅仅连接输入单元的一部分
 - 一组连接可以共享同一个权重
 - 池化层（Pooling layer）
 - 将卷积后的特征切成几个区域，取其最大值或平均值
 - 得到新的、维度较小的特征，从而减少隐层结点数。
 - 通过全连接层把所有局部特征结合变成全局特征，输出识别或分类结果。

卷积神经网络 (CNN)

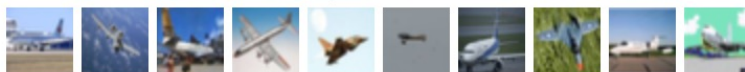
- 无需对图像进行复杂的前期预处理
- 自动筛选出有利于分类的局部特征



用Keras实现图像分类

- 图像数据集CIFAR-10
 - 包括6万张 32×32 的 RGB 彩色图像图片
 - 被分成10类
 - 5万张图片用于训练， 1万张图片用于测试。

airplane



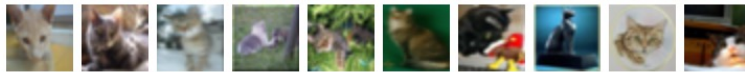
automobile



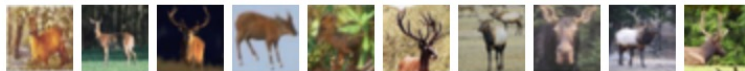
bird



cat



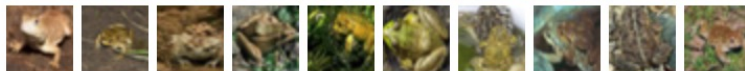
deer



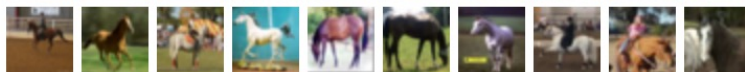
dog



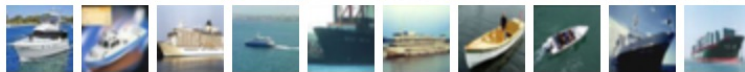
frog



horse



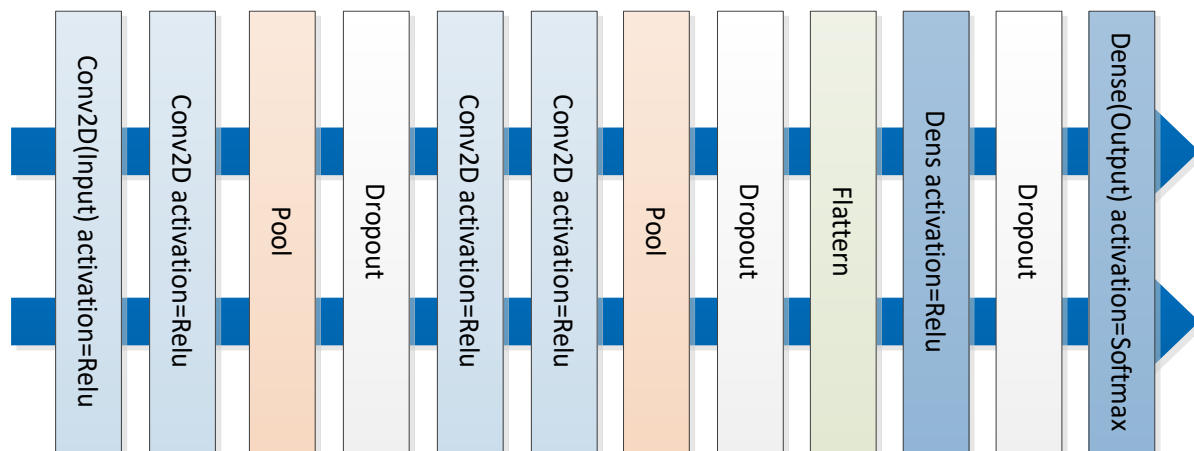
ship



truck



图像分类的实现



- 构建CNN网络

- 基础的神经网络层
- 卷积层Conv1D、Conv2D
 - Conv2D(filters, kernel_size, strides=(1,1), padding='valid',...)
- 池化层 MaxPooling2D、AveragePooling2D
 - MaxPooling2D(pool_size=(2, 2), ...)
- Flatten层

例7-2：构建CNN深度神经网络，基于CIFAR-10训练图像分类器

1) 读取CIFAR-10数据集

- Keras自带，可通过load_data()函数获得

```
from keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

2) 对数据进行预处理

- 将图像的像素值归一化为[0,1]之间
- 多分类问题，需要将类标签向量转换为二元类矩阵

```
num_classes = 10 #分类个数
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255
y_train = np_utils.to_categorical(y_train, num_classes)
y_test = np_utils.to_categorical(y_test, num_classes)
```

3) 构建CNN模型

```
from keras.layers import Dense, Dropout, Activation, Flatten
from keras.layers import Conv2D, MaxPooling2D
model = Sequential()
model.add(Conv2D(32, (3, 3), padding='same',
                 input_shape=x_train.shape[1:]))
model.add(Activation('relu'))
model.add(Conv2D(32, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))

model.add(Conv2D(64, (3, 3), padding='same'))
model.add(Activation('relu'))
model.add(Conv2D(64, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))

model.add(Flatten())
model.add(Dense(512))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes))
model.add(Activation('softmax'))
```

4) 对构建好的CNN模型进行编译

- 因为是多分类问题，所以选择损失函数
“categorical_crossentropy”，同时优化器选择 “rmsprop”

```
# 初始化RMSprop优化器
opt = keras.optimizers.rmsprop(lr=0.0001, decay=1e-6)
# 模型编译
model.compile(loss='categorical_crossentropy',
optimizer=opt, metrics=['accuracy'])
```

5) 训练 CNN模型

```
batch_size = 32
epochs = 100 #参数学习算法的迭代次数
model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,
validation_data=(x_test, y_test), shuffle=True)
```

6) 对 CNN模型进行性能评估

```
scores = model.evaluate(x_test, y_test, verbose=1)
print('Test loss:', scores[0])
print('Test accuracy:', scores[1])
```

使用预训练模型进行图像分类

- 模型训练是一项非常耗时
- Keras也包含了很多目前非常优秀的预训练模型
 - 如Xception、VGG16、VGG19等
 - 模型在ImageNet图像数据集 (<http://www.image-net.org/>) 上训练获得的
- 使用Keras的ResNet50图像分类模型预测大象图片分类



使用预训练模型进行图像分类

```
from keras.applications.resnet50 import ResNet50
from keras.applications.resnet50 import preprocess_input
from keras.applications.resnet50 import
decode_predictions
from keras.preprocessing import image
import numpy as np

#导入预训练模型ResNet50
model = ResNet50(weights='imagenet')

#对输入图片进行处理
img_path = 'data/elephant.jpg'
img = image.load_img(img_path, target_size=(224, 224))
X = image.img_to_array(img) #将图像转换为数组
X = np.expand_dims(X, axis=0)
X = preprocess_input(x)

#模型预测
preds = model.predict(X)
print('Predicted:', decode_predictions(preds, top=3)[0])
```

- 输出预测结果，按照概率排序前3的类型为：Indian_elephant（印度象）、tusker（有长牙的动物）和African_elephant（非洲象）。

思考与练习

- 1.设计不同的CNN模型训练CIFAR-10图像集，比较不同模型下图像分类的效果。
- 2.选择不同的优化器和损失函数，比较图像分类的效果。